

Functional Programming

Algebraic datatypes

Jesper Bengtson

Last weeks

We covered a substantial part of F#

- Higher-order functions
- Function composition
- Piping commands
- More lists

Folds (loops)

The following Java program sums all elements of a list

```
public static int sumList(List<Integer> input) {  
    int accumulator = 0;  
    for(int el : input) { accumulator += el; }  
    return accumulator;  
}
```


Folds (loops)

The following Java program sums all elements of a list

```
public static int sumList(List<Integer> input) {  
    int accumulator = 0;  
    for(int el : input) { accumulator += el; }  
    return accumulator;  
}
```

and the following does the same in F#

```
let rec sumList =  
    function  
    | []      -> 0  
    | x::xs  -> x + sumList xs
```

Folds (loops)

The following Java program sums all elements of a list

```
public static int sumList(List<Integer> input) {
    int accumulator = 0;
    for(int el : input) { accumulator += el; }
    return accumulator;
}
```

Another version that follows the loop a bit more closely

```
let sumList input =
  let rec loop accumulator =
    function
      | []      -> accumulator
      | el::xs  -> loop (accumulator + el) xs
  loop 0 input
```

Folds (loops)

```
let sumList input =  
  let rec loop accumulator =  
    function  
      | []      -> accumulator  
      | e1::xs  -> loop (accumulator + e1) xs  
  loop 0 input
```

More generally

```
let rec fold folder accumulator =  
  function  
    | []      -> accumulator  
    | e1::xs  -> fold folder (folder accumulator e1) xs
```


Folds (loops)

```
let sumList input =  
  let rec loop accumulator =  
    function  
      | []      -> accumulator  
      | e1::xs  -> loop (accumulator + e1) xs  
  loop 0 input
```

More generally

```
let rec fold folder accumulator =  
  function  
    | []      -> accumulator  
    | e1::xs  -> fold folder (folder accumulator e1) xs
```

```
sumList lst = fold (fun a b -> a + b) 0 lst
```

Folds (loops)

```
let rec fold folder accumulator =  
  function  
  | []      -> accumulator  
  | e1::xs  -> fold folder (folder accumulator e1) xs
```

```
sumList lst = fold (fun a b -> a + b) 0 lst
```

$\text{sumList } [1;2;3;4;5] = (((((0 + 1) + 2) + 3) + 4) + 5) = 15$

Folds (loops)

```
let rec fold folder accumulator =  
  function  
  | []      -> accumulator  
  | el::xs  -> fold folder (folder accumulator el) xs
```

```
fold : ('b -> 'a -> 'b) -> 'b -> 'a list -> 'b
```

Folds (loops)

```
let rec fold folder accumulator =
  function
  | []      -> accumulator
  | e1::xs  -> fold folder (folder accumulator e1) xs
```

`fold : ('b -> 'a -> 'b) -> 'b -> 'a list -> 'b`

Mathematically

$$\text{fold } f \text{ acc } [x_1; x_2; \dots; x_n]$$

returns

$$f \text{ } (\dots (f \text{ } (f \text{ acc } x_1) \text{ } x_2) \dots x_{n-1}) \text{ } x_n$$

Folds (evaluation)

```
let rec fold folder accumulator =  
  function  
  | []      -> accumulator  
  | x :: xs -> fold folder (folder accumulator x) xs  
  
fold (-) 0 [1; 2; 3]
```


Folds (evaluation)

```
let rec fold folder accumulator =  
  function  
  | []          -> accumulator  
  | x :: xs     -> fold folder (folder accumulator x) xs  
  
fold (-) 0 [1; 2; 3] ~  
fold (-) ((-) 0 1) [2; 3]
```

Folds (evaluation)

```
let rec fold folder accumulator =  
  function  
  | []      -> accumulator  
  | x :: xs -> fold folder (folder accumulator x) xs
```

`fold (-) 0 [1; 2; 3]  $\leadsto$`

`fold (-) ((-) 0 1) [2; 3]  $\leadsto$`

`fold (-) (0 - 1) [2; 3]`

Folds (evaluation)

```
let rec fold folder accumulator =  
  function  
  | []      -> accumulator  
  | x :: xs -> fold folder (folder accumulator x) xs
```

`fold (-) 0 [1; 2; 3]  $\leadsto$`

`fold (-) ((-) 0 1) [2; 3]  $\leadsto$`

`fold (-) (0 - 1) [2; 3]  $\leadsto$  fold (-) -1 [2; 3]`

Folds (evaluation)

```
let rec fold folder accumulator =  
  function  
  | []      -> accumulator  
  | x :: xs -> fold folder (folder accumulator x) xs
```

`fold (-) 0 [1; 2; 3]  $\leadsto$`

`fold (-) ((-) 0 1) [2; 3]  $\leadsto$`

`fold (-) (0 - 1) [2; 3]  $\leadsto$  fold (-) -1 [2; 3]  $\leadsto$`

`fold (-) ((-) -1 2) [3]`

Folds (evaluation)

```
let rec fold folder accumulator =
  function
  | []          -> accumulator
  | x :: xs     -> fold folder (folder accumulator x) xs
```

`fold (-) 0 [1; 2; 3]  $\leadsto$`

`fold (-) ((-) 0 1) [2; 3]  $\leadsto$`

`fold (-) (0 - 1) [2; 3]  $\leadsto$  fold (-) -1 [2; 3]  $\leadsto$`

`fold (-) ((-) -1 2) [3]  $\leadsto$`

`fold (-) (-1 - 2) [3]`

Folds (evaluation)

```
let rec fold folder accumulator =  
  function  
  | []      -> accumulator  
  | x :: xs -> fold folder (folder accumulator x) xs
```

`fold (-) 0 [1; 2; 3]  $\leadsto$`

`fold (-) ((-) 0 1) [2; 3]  $\leadsto$`

`fold (-) (0 - 1) [2; 3]  $\leadsto$  fold (-) -1 [2; 3]  $\leadsto$`

`fold (-) ((-) -1 2) [3]  $\leadsto$`

`fold (-) (-1 - 2) [3]  $\leadsto$  fold (-) -3 [3]`

Folds (evaluation)

```

let rec fold folder accumulator =
  function
  | []          -> accumulator
  | x :: xs    -> fold folder (folder accumulator x) xs

fold (-) 0 [1; 2; 3] ~>
fold (-) ((-) 0 1) [2; 3] ~>
fold (-) (0 - 1) [2; 3] ~> fold (-) -1 [2; 3] ~>
fold (-) ((-) -1 2) [3] ~>
fold (-) (-1 - 2) [3] ~> fold (-) -3 [3] ~>
fold (-) ((-) -3 3) []

```

Folds (evaluation)

```

let rec fold folder accumulator =
  function
  | []          -> accumulator
  | x :: xs     -> fold folder (folder accumulator x) xs

fold (-) 0 [1; 2; 3] ~>
fold (-) ((-) 0 1) [2; 3] ~>
fold (-) (0 - 1) [2; 3] ~> fold (-) -1 [2; 3] ~>
fold (-) ((-) -1 2) [3] ~>
fold (-) (-1 - 2) [3] ~> fold (-) -3 [3] ~>
fold (-) ((-) -3 3) [] ~>
fold (-) (-3 - 3) []

```

Folds (evaluation)

```
let rec fold folder accumulator =  
  function  
  | []      -> accumulator  
  | x :: xs -> fold folder (folder accumulator x) xs  
  
fold (-) 0 [1; 2; 3] ~  
fold (-) ((-) 0 1) [2; 3] ~  
fold (-) (0 - 1) [2; 3] ~ fold (-) -1 [2; 3] ~  
fold (-) ((-) -1 2) [3] ~  
fold (-) (-1 - 2) [3] ~ fold (-) -3 [3] ~  
fold (-) ((-) -3 3) [] ~  
fold (-) (-3 - 3) [] ~ fold (-) -6 []
```


Folds (evaluation)

```

let rec fold folder accumulator =
  function
  | []          -> accumulator
  | x :: xs    -> fold folder (folder accumulator x) xs

fold (-) 0 [1; 2; 3] ~>
fold (-) ((-) 0 1) [2; 3] ~>
fold (-) (0 - 1) [2; 3] ~> fold (-) -1 [2; 3] ~>
fold (-) ((-) -1 2) [3] ~>
fold (-) (-1 - 2) [3] ~> fold (-) -3 [3] ~>
fold (-) ((-) -3 3) [] ~>
fold (-) (-3 - 3) [] ~> fold (-) -6 [] ~>
-6

```


Quote from student towards the end of the course
(paraphrased)

"I just realised that with folds you can do absolutely anything!"

Quote from student towards the end of the course
(paraphrased)

"I just realised that with folds you can do absolutely anything!"

Not quite true, but the TAs were dancing

Example

Create a function that given a list of integers creates a string containing the last digit of all of the odd elements

Example

Create a function that given a list of integers creates a string containing the last digit of all of the odd elements

- Remove all even numbers
- Obtain the last digit from each number
- Construct a string containing the last digit of each number

What higher-order functions should we use for these steps?

Example

Create a function that given a list of integers creates a string containing the last digit of all of the odd elements

- Remove all even numbers filter
- Obtain the last digit from each number map
- Construct a string containing the last digit of each number fold

Example

Create a function that given a list of integers creates a string containing the last digit of all of the odd elements

- Remove all even numbers filter
- Obtain the last digit from each number map
- Construct a string containing the last digit of each number fold

```
mergeLastDigits [123; 124; 5; 1233] = [123; 5; 1233]
```

```
let mergeLastDigits lst =
```

```
(filter (fun x -> x % 2 = 1) lst)
```


Example

Create a function that given a list of integers creates a string containing the last digit of all of the odd elements

- Remove all even numbers filter
- Obtain the last digit from each number map
- Construct a string containing the last digit of each number fold

```
mergeLastDigits [123; 124; 5; 1233] = [3; 5; 3]
```

```
let mergeLastDigits lst =
```

```
    (map (fun x -> x % 10)  
      (filter (fun x -> x % 2 = 1) lst))
```

Example

Create a function that given a list of integers creates a string containing the last digit of all of the odd elements

- Remove all even numbers filter
- Obtain the last digit from each number map
- Construct a string containing the last digit of each number fold

```
mergeLastDigits [123; 124; 5; 1233] = "353"  
let mergeLastDigits lst =  
  fold (fun acc x -> acc + string x) ""  
    (map (fun x -> x % 10)  
      (filter (fun x -> x % 2 = 1) lst))
```

Example

```
let mergeLastDigits lst =  
  fold (fun acc x -> acc + string x) ""  
    (map (fun x -> x % 10)  
      (filter (fun x -> x % 2 = 1) lst))
```

which is the same as

```
let mergeLastDigits lst =  
  lst |>  
  filter (fun x -> x % 2 = 1) |>  
  map (fun x -> x % 10) |>  
  fold (fun acc x -> acc + string x) ""
```


Example

```
let mergeLastDigits lst =  
  fold (fun acc x -> acc + string x) ""  
    (map (fun x -> x % 10)  
      (filter (fun x -> x % 2 = 1) lst))
```

which is the same as

```
let mergeLastDigits lst =  
  lst |>  
  filter (fun x -> x % 2 = 1) |>  
  map (fun x -> x % 10) |>  
  fold (fun acc x -> acc + string x) ""
```

This is MUCH easier to read

Example

```
let mergeLastDigits lst =  
  lst |>  
  filter (fun x -> x % 2 = 1) |>  
  map (fun x -> x % 10) |>  
  fold (fun acc x -> acc + string x) ""
```

which is the same as

```
let mergeLastDigits =  
  filter (fun x -> x % 2 = 1) >>  
  map (fun x -> x % 10) >>  
  fold (fun acc x -> acc + string x) ""
```


Example

```
let mergeLastDigits =  
  filter (fun x -> x % 2 = 1) >>  
  map (fun x -> x % 10) >>  
  fold (fun acc x -> acc + string x) ""
```

which is the same as

```
let mergeLastDigits =  
  filter (fun x -> x % 2 = 1) >>  
  map (fun x -> x % 10) >>  
  fold (fun acc x -> x |> string |> (+) acc) ""
```


Example

```
let mergeLastDigits =  
  filter (fun x -> x % 2 = 1) >>  
  map (fun x -> x % 10) >>  
  fold (fun acc x -> x |> string |> (+) acc) ""
```

which is the same as

Which is not easier to read

```
let mergeLastDigits =  
  filter (fun x -> x % 2 = 1) >>  
  map (fun x -> x % 10) >>  
  fold (fun acc -> string >> (+) acc) ""
```

And the winner is....

In my highly subjective opinion... one of these two

```
let mergeLastDigits lst =  
  lst |>  
  filter (fun x -> x % 2 = 1) |>  
  map (fun x -> x % 10) |>  
  fold (fun acc x -> acc + string x) ""
```

```
let mergeLastDigits =  
  filter (fun x -> x % 2 = 1) >>  
  map (fun x -> x % 10) >>  
  fold (fun acc x -> acc + string x) ""
```

Questions?

Things to practice on

Become good friends with

- Piping (`|>`)
- Function composition (`>>`)
- `map`
- `filter`
- `fold`

This week

- Inductively defined datatypes
- The Option datatype
- The Result datatype
- Expression trees
- Data representation
- Possibly live coding

Inductively defined types

- ... or algebraic datatypes
- ... or disjoint unions

Allow us to concisely say how
members of types are created

Inductively defined types

```
type month =  
| January | February | March | April  
| May | June | July | August  
| September | October | November  
| December
```

```
type weekDay =  
| Monday | Tuesday | Wednesday  
| Thursday | Friday |  
| Saturday | Sunday
```

Inductively defined types

```
type month =  
  | January | February | March | April
```

Observation 1

We use 'type', not 'let'

```
type weekday =  
  | Monday | Tuesday | Wednesday  
  | Thursday | Friday |  
  | Saturday | Sunday
```

Inductively defined types

```
type month =  
  | January | February | March | April
```

Observation 2

Behave similarly to enum-types in Java

```
type weekday =  
  | Monday | Tuesday | Wednesday  
  | Thursday | Friday |  
  | Saturday | Sunday
```


Programming

```
type month =  
  | January | February | March | April  
  | May | June | July | August  
  | September | October | November  
  | December  
  
let numberOfDays =  
function  
  | January | March | May | July  
  | August | October | December -> 31  
  | February -> 28  
  | _ -> 30
```


Programming

We can pattern match on algebraic types

```
let numberOfDays =  
function  
| January | March | May | July  
| August | October | December -> 31  
| February -> 28  
| _ -> 30
```

Programming

```
type weekDay =  
  | Monday | Tuesday | Wednesday  
  | Thursday | Friday |  
  | Saturday | Sunday  
  
let nextWeekday =  
  function  
    | Monday      -> Tuesday  
    | Tuesday     -> Wednesday  
    | Wednesday   -> Thursday  
    | Thursday    -> Friday  
    | _           -> Monday
```


Arguments

Algebraic types can take arguments

```
type shape =  
  | Circ of float (* radius *)  
  | Rect of float * float (* sides *)  
  
let circumference =  
  function  
    | Circ r      -> 2 * System.Math.PI * r  
    | Rect (w, h) -> 2 * (w + h)
```

Arguments

Algebraic types can take arguments

```
type shape =  
  | Circ of float          (* radius *)  
  | Rect of float * float  (* sides *)  
  
let area =  
  function  
    | Circ r          -> System.Math.PI * r * r  
    | Rect (w, h)     -> w * h
```

The option type

Options are used to encode
partial functions

```
type 'a option =  
| None          (* No result *)  
| Some of 'a    (* result *)
```

You can think of options as terms that can
be set to null (but nice, and type-safe,
and does not cause as many bugs)

Head and tails of a list

The functions `List.head` and `List.tail` will throw an exception if you try to get the head of an empty list (`List.head []` or `List.tail []`)

```
let safeHead =  
  function  
  | []      -> None  
  | x :: _  -> Some x
```

```
let safeTail =  
  function  
  | []      -> None  
  | _ :: xs -> Some xs
```

Some useful functions

`Option.get : 'a option -> 'a`

`Option.map : ('a -> 'b) -> 'a option -> 'b option`

`Option.defaultValue : 'a -> 'a option -> 'a`

`let get =`

`function`

`| Some x -> x`

`let defaultValue v =`

`function`

`| None -> v`

`| Some x -> x`

`let map f =`

`function`

`| None -> None`

`| Some x -> Some (f x)`

Some useful functions

`Option.get : 'a option -> 'a`

`Option.map : ('a -> 'b) -> 'a option -> 'b option`

`Option.defaultValue : 'a -> 'a option -> 'a`

```
let get =
  function
  | Some x -> x
```

```
let defaultValue v =
  function
  | None -> v
  | Some x -> x
```

```
let map f =
  function
  | None -> None
  | Some x -> Some (f x)
```

`Option.get` `None` throws an exception, similarly to taking the head of an empty list

Examples

```
let safeHead =
```

```
function
```

```
| []      -> None  
| x :: _  -> Some x
```

```
let safeTail =
```

```
function
```

```
| []      -> None  
| _ :: xs -> Some xs
```

```
Option.get : 'a option -> 'a
```

```
Option.map : ('a -> 'b) -> 'a option -> 'b option
```

```
Option.defaultValue : 'a -> 'a option -> 'a
```

```
Option.get (safeHead [1;2;3]) = 1
```

which is the same as

```
[1;2;3] |> safeHead |> Option.get = 1
```

Examples

```
let safeHead =
```

```
function
```

```
| []      -> None  
| x :: _  -> Some x
```

```
let safeTail =
```

```
function
```

```
| []      -> None  
| _ :: xs -> Some xs
```

```
Option.get : 'a option -> 'a
```

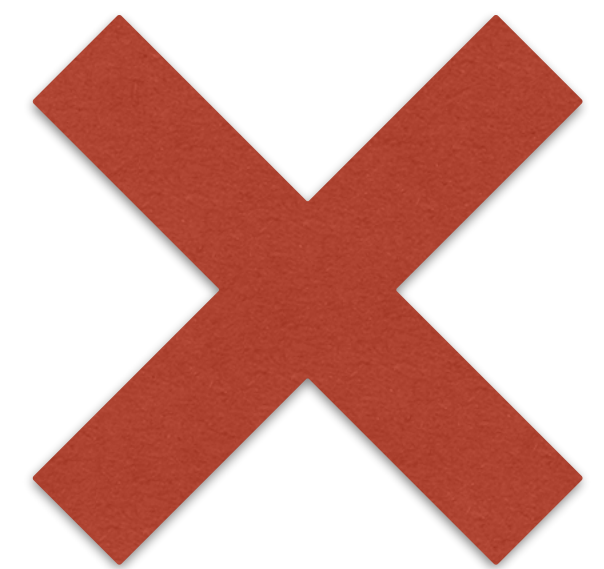
```
Option.map : ('a -> 'b) -> 'a option -> 'b option
```

```
Option.defaultValue : 'a -> 'a option -> 'a
```

```
Option.get (safeHead [])
```

which is the same as

```
[] |> safeHead |> Option.get
```



Examples

```
let safeHead =
```

```
function
```

```
| []      -> None  
| x :: _  -> Some x
```

```
let safeTail =
```

```
function
```

```
| []      -> None  
| _ :: xs -> Some xs
```

```
Option.get : 'a option -> 'a
```

```
Option.map : ('a -> 'b) -> 'a option -> 'b option
```

```
Option.defaultValue : 'a -> 'a option -> 'a
```

```
Option.get (safeTail [1;2;3]) = [2; 3]
```

which is the same as

```
[1;2;3] |> safeTail |> Option.get = [2; 3]
```


Examples

```
let safeHead =
```

```
function
```

```
| []      -> None  
| x :: _  -> Some x
```

```
let safeTail =
```

```
function
```

```
| []      -> None  
| _ :: xs -> Some xs
```

```
Option.get : 'a option -> 'a
```

```
Option.map : ('a -> 'b) -> 'a option -> 'b option
```

```
Option.defaultValue : 'a -> 'a option -> 'a
```

```
Option.map (fun x -> x + 10) (safeHead [1;2;3]) = Some 11
```

which is the same as

```
[1;2;3] |> safeHead |> Option.map (fun x -> x + 10) = Some 11
```

Examples

```
let safeHead =
```

```
function
```

```
| []      -> None  
| x :: _  -> Some x
```

```
let safeTail =
```

```
function
```

```
| []      -> None  
| _ :: xs -> Some xs
```

```
Option.get : 'a option -> 'a
```

```
Option.map : ('a -> 'b) -> 'a option -> 'b option
```

```
Option.defaultValue : 'a -> 'a option -> 'a
```

```
Option.map (fun x -> x + 10) (safeHead []) = None
```

which is the same as

```
[] |> safeHead |> Option.map (fun x -> x + 10) = None
```


Examples

```
let safeHead =
```

```
function
```

```
| []      -> None  
| x :: _  -> Some x
```

```
let safeTail =
```

```
function
```

```
| []      -> None  
| _ :: xs -> Some xs
```

```
Option.get : 'a option -> 'a
```

```
Option.map : ('a -> 'b) -> 'a option -> 'b option
```

```
Option.defaultValue : 'a -> 'a option -> 'a
```

```
defaultValue 0 (safeHead [1;2;3]) = 1
```

which is the same as

```
[1;2;3] |> safeHead |> defaultValue 0 = 1
```


Examples

```
let safeHead =
```

```
function
```

```
| []      -> None  
| x :: _  -> Some x
```

```
let safeTail =
```

```
function
```

```
| []      -> None  
| _ :: xs -> Some xs
```

```
Option.get : 'a option -> 'a
```

```
Option.map : ('a -> 'b) -> 'a option -> 'b option
```

```
Option.defaultValue : 'a -> 'a option -> 'a
```

```
defaultValue 0 (safeHead []) = 0
```

which is the same as

```
[] |> safeHead |> defaultValue 0 = 0
```

Examples

What if you want to get the third element from a list?

```
[1;2;3;4] |> List.tail |> List.tail |> List.head = 3
```

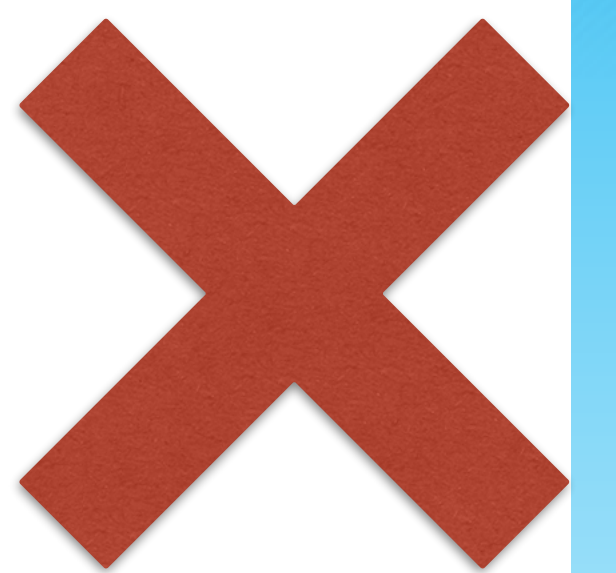
Examples

What if you want to get the third element from a list?

```
[1;2;3;4] |> List.tail |> List.tail |> List.head = 3
```

But this is not safe

```
[1;2] |> List.tail |> List.tail |> List.head =
```



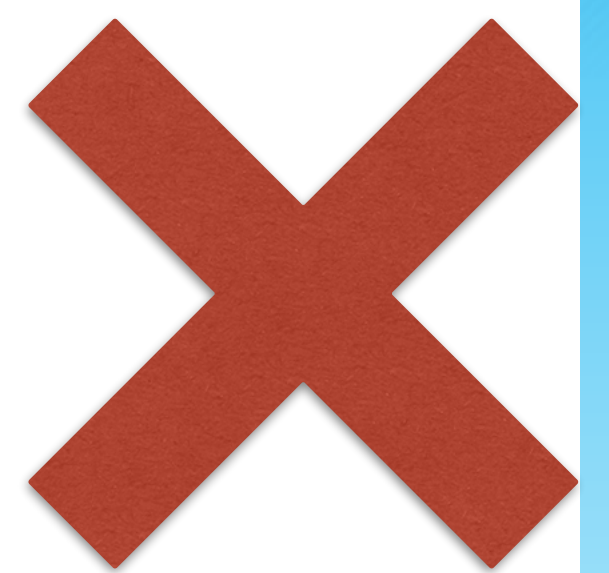
Examples

What if you want to get the third element from a list?

```
[1;2;3;4] |> List.tail |> List.tail |> List.head = 3
```

But this is not safe

```
[1;2] |> List.tail |> List.tail |> List.head =
```



We can't get the head of the empty list

Examples

```
let safeHead =
```

```
function
```

```
| []          -> None
```

```
| x :: _      -> Some x
```

```
let safeTail =
```

```
function
```

```
| []          -> None
```

```
| _ :: xs     -> Some xs
```

```
[1;2;3;4] |> safeTail |> safeTail |> safeHead = ???
```


Examples

```
let safeHead =
```

```
function
```

```
| []      -> None
```

```
| x :: _  -> Some x
```

```
let safeTail =
```

```
function
```

```
| []      -> None
```

```
| _ :: xs -> Some xs
```

```
[1;2;3;4] |> safeTail |> safeTail |> safeHead = ???
```

Does this even type check?

Examples

```
let safeHead =
```

```
function
```

```
| [] -> None
```

```
| x :: _ -> Some x
```

```
let safeTail =
```

```
function
```

```
| [] -> None
```

```
| _ :: xs -> Some xs
```

```
[1;2;3;4] |> safeTail |> safeTail |> safeHead = ???
```

```
[1;2;3;4]
```

Examples

```
let safeHead =
```

```
function
```

```
| []      -> None
```

```
| x :: _  -> Some x
```

```
let safeTail =
```

```
function
```

```
| []      -> None
```

```
| _ :: xs -> Some xs
```

```
[1;2;3;4] |> safeTail |> safeTail |> safeHead = ???
```

```
Some [2;3;4]
```


Examples

```
let safeHead =
```

```
function
```

```
| []      -> None
```

```
| x :: _  -> Some x
```

```
let safeTail =
```

```
function
```

```
| []      -> None
```

```
| _ :: xs -> Some xs
```

```
[1;2;3;4] |> safeTail |> safeTail
```

Some [2;3;4]

Type error! expects `int list`, but receives an `int list option`

Examples

```
let safeHead =  
  function  
  | []      -> None  
  | x :: _  -> Some x
```

```
let safeTail =  
  function  
  | []      -> None  
  | _ :: xs -> Some xs
```

```
let safeHead2 =  
  function  
  | Some (x :: _) -> Some x  
  | _            -> None
```

```
let safeTail2 =  
  function  
  | Some (_ :: xs) -> Some xs  
  | _              -> None
```

The inputs have type `int list option`
rather than `int list`

Examples

```
let safeHead =  
  function  
  | []      -> None  
  | x :: _  -> Some x
```

```
let safeTail =  
  function  
  | []      -> None  
  | _ :: xs -> Some xs
```

```
let safeHead2 =  
  function  
  | Some (x :: _) -> Some x  
  | _             -> None
```

```
let safeTail2 =  
  function  
  | Some (_ :: xs) -> Some xs  
  | _              -> None
```

```
[1;2;3;4] |> safeTail |> safeTail2 |> safeHead2 =  
          Some 3
```

Examples

```
let safeHead =  
  function  
  | []      -> None  
  | x :: _  -> Some x
```

```
let safeTail =  
  function  
  | []      -> None  
  | _ :: xs -> Some xs
```

```
let safeHead2 =  
  function  
  | Some (x :: _) -> Some x  
  | _             -> None
```

```
let safeTail2 =  
  function  
  | Some (_ :: xs) -> Some xs  
  | _              -> None
```

```
[1;2] |> safeTail |> safeTail2 |> safeHead2 =  
      None
```

Examples

```
let safeHead =  
  function  
  | []      -> None  
  | x :: _  -> Some x
```

```
let safeTail =  
  function  
  | []      -> None  
  | _ :: xs -> Some xs
```

```
let safeHead2 =  
  function  
  | Some (x :: _) -> Some x  
  | _             -> None
```

```
let safeTail2 =  
  function  
  | Some (_ :: xs) -> Some xs  
  | _              -> None
```

This works, but it is very unsatisfying.
We need double functions for anything where we
want to pipe options together

Bind

We have seen these before

```
Option.get : 'a option -> 'a
```

```
Option.map : ('a -> 'b) -> 'a option -> 'b option
```

```
Option.defaultValue : 'a -> 'a option -> 'a
```

But not one of the most useful ones

```
Option.bind :
```

```
  ('a -> 'b option) -> 'a option -> 'b option
```


Bind

```
Option.bind :  
  ('a -> 'b option) -> 'a option -> 'b option  
  
let bind binder =  
  function  
  | None    -> None  
  | Some x  -> binder x
```

The bind function unfolds the option. The intuition is that if a result ever becomes None, it will stay there.

Bind

```
Option.bind :
  ('a -> 'b option) ->
  'a option ->
  'b option
```

```
let safeHead =
  function
  | []      -> None
  | x :: _ -> Some x
```

```
let bind binder =
  function
  | None      -> None
  | Some x    -> binder x
```

```
let safeTail =
  function
  | []      -> None
  | _ :: xs -> Some xs
```

Bind

```
Option.bind :
  ('a -> 'b option) ->
  'a option ->
  'b option
```

```
let safeHead =
  function
  | []      -> None
  | x :: _ -> Some x
```

```
let bind binder =
  function
  | None      -> None
  | Some x    -> binder x
```

```
let safeTail =
  function
  | []      -> None
  | _ :: xs -> Some xs
```

```
[1;2;3;4;5] |> safeTail |> Option.bind safeTail |>
Option.bind safeTail |> Option.bind safeHead =
Some 4
```

Bind

```
Option.bind :
  ('a -> 'b option) ->
  'a option ->
  'b option
```

```
let safeHead =
  function
  | []      -> None
  | x :: _  -> Some x
```

```
let bind binder =
  function
  | None      -> None
  | Some x    -> binder x
```

```
let safeTail =
  function
  | []      -> None
  | _ :: xs -> Some xs
```

```
[1;2;3;4;5] |> safeTail |> Option.bind safeTail |>
Option.bind safeTail |> Option.bind safeHead =
[1;2;3;4;5]
```


Bind

```
Option.bind :
  ('a -> 'b option) ->
  'a option ->
  'b option
```

```
let safeHead =
  function
  | []      -> None
  | x :: _ -> Some x
```

```
let binder =
  function
  | None      -> None
  | Some x    -> binder x
```

```
let safeTail =
  function
  | []      -> None
  | _ :: xs -> Some xs
```

```
[1;2;3;4;5] |> safeTail |> Option.bind safeTail |>
Option.bind safeTail |> Option.bind safeHead =
Some [2;3;4;5]
```

Bind

```

Option.bind :
  ('a -> 'b option) ->
  'a option ->
  'b option

let bind binder =
  function
  | None    -> None
  | Some x -> binder x

let safeHead =
  function
  | []      -> None
  | x :: _ -> Some x

let safeTail =
  function
  | []      -> None
  | _ :: xs -> Some xs

[1;2;3;4;5] |> safeTail |> Option.bind safeTail |>
Option.bind safeTail |> Option.bind safeHead =
Some [3;4;5]

```


Bind

```
Option.bind :
  ('a -> 'b option) ->
  'a option ->
  'b option
```

```
let safeHead =
  function
  | []      -> None
  | x :: _ -> Some x
```

```
let bind binder =
  function
  | None      -> None
  | Some x    -> binder x
```

```
let safeTail =
  function
  | []      -> None
  | _ :: xs -> Some xs
```

```
[1;2;3;4;5] |> safeTail |> Option.bind safeTail |>
Option.bind safeTail |> Option.bind safeHead =
Some [4;5]
```

Bind

```
Option.bind :
  ('a -> 'b option) ->
  'a option ->
  'b option
```

```
let bind binder =
  function
  | None    -> None
  | Some x  -> binder x
```

```
let safeHead =
  function
  | []      -> None
  | x :: _  -> Some x
```

```
let safeTail =
  function
  | []      -> None
  | _ :: xs -> Some xs
```

```
[1;2;3;4;5] |> safeTail |> Option.bind safeTail |>
Option.bind safeTail |> Option.bind safeHead =
Some 4
```

Bind

```

Option.bind :
  ('a -> 'b option) ->
  'a option ->
  'b option

let bind binder =
  function
  | None    -> None
  | Some x -> binder x

let safeHead =
  function
  | []      -> None
  | x :: _ -> Some x

let safeTail =
  function
  | []      -> None
  | _ :: xs -> Some xs

[1;2] |> safeTail |> Option.bind safeTail |>
Option.bind safeTail |> Option.bind safeHead =
None

```


Bind

```

Option.bind :
  ('a -> 'b option) ->
  'a option ->
  'b option

let bind binder =
  function
  | None    -> None
  | Some x -> binder x

let safeHead =
  function
  | []      -> None
  | x :: _ -> Some x

let safeTail =
  function
  | []      -> None
  | _ :: xs -> Some xs

```

```

[1;2] |> safeTail |> Option.bind safeTail |>
Option.bind safeTail |> Option.bind safeHead =
[1;2]

```

Bind

```
Option.bind :
  ('a -> 'b option) ->
  'a option ->
  'b option
```

```
let safeHead =
  function
  | []      -> None
  | x :: _ -> Some x
```

```
let bind binder =
  function
  | None      -> None
  | Some x    -> binder x
```

```
let safeTail =
  function
  | []      -> None
  | _ :: xs -> Some xs
```

```
[1;2] |> safeTail |> Option.bind safeTail |>
Option.bind safeTail |> Option.bind safeHead =
Some [2]
```


Bind

```

Option.bind :
  ('a -> 'b option) ->
  'a option ->
  'b option

let bind binder =
  function
  | None    -> None
  | Some x -> binder x

let safeHead =
  function
  | []      -> None
  | x :: _ -> Some x

let safeTail =
  function
  | []      -> None
  | _ :: xs -> Some xs

[1;2] |> safeTail |> Option.bind safeTail |>
Option.bind safeTail |> Option.bind safeHead =
Some []

```

Bind

```

Option.bind :
  ('a -> 'b option) ->
  'a option ->
  'b option

let bind binder =
  function
  | None    -> None
  | Some x -> binder x

let safeHead =
  function
  | []      -> None
  | x :: _ -> Some x

let safeTail =
  function
  | []      -> None
  | _ :: xs -> Some xs

[1;2] |> safeTail |> Option.bind safeTail |>
Option.bind safeTail |> Option.bind safeHead =
None

```

Bind

```

Option.bind :
  ('a -> 'b option) ->
  'a option ->
  'b option

let bind binder =
  function
  | None    -> None
  | Some x -> binder x

let safeHead =
  function
  | []      -> None
  | x :: _ -> Some x

let safeTail =
  function
  | []      -> None
  | _ :: xs -> Some xs

[1;2] |> safeTail |> Option.bind safeTail |>
Option.bind safeTail |> Option.bind safeHead =
None

```


Bind

```
Option.bind :           let bind binder =
  ('a -> 'b option) ->   function
  'a option ->           | None    -> None
```

The bind function unfolds the option. The intuition is that if a result ever becomes None, it will stay there.

```
function
| []      -> None
| x :: _ -> Some x
```

```
function
| []      -> None
| _ :: xs -> Some xs
```

```
[1;2] |> safeTail |> Option.bind safeTail |>
Option.bind safeTail |> Option.bind safeHead =
None
```


Bind

```
Option.bind :           let bind binder =
  ('a -> 'b option) ->   function
  'a option ->           | None    -> None
```

The bind function unfolds the option. The intuition is that if a result ever becomes None, it will stay there.

The function safeHead is never even called!

```
[ ... ]
Option.bind safeTail |> Option.bind safeHead =
None
```


Bind is universal

When we come to railroad oriented programming (monads) you will use bind a lot so get used to it. It's an amazing abstraction and we are just scratching the surface here.

Questions?

The Result datatype

Options either succeed or fail, but they will never tell you why they failed

The result type helps

```
type Result<'a, 'b> =  
| Ok of 'a          (* result *)  
| Error of 'b        (* error *)
```


Examples

```
type Result<'a, 'b> =  
    | Ok of 'a          (* result *)  
    | Error of 'b       (* error *)  
  
let safeHead =  
    function  
    | []      -> Error "Cannot take the head of  
                      an empty list"  
    | x :: _  -> Ok x  
  
let safeTail =  
    function  
    | []      -> Error "Cannot take the tail of an  
                      Empty list"  
    | _ :: xs -> Ok xs
```

Examples

```
let safeHead =  
  function  
  | []      -> Error "Cannot take the head of  
                    an empty list"  
  | x :: _  -> Ok x  
let safeTail =  
  function  
  | []      -> Error "Cannot take the tail of an  
                    Empty list"  
  | _ :: xs -> Ok xs
```

```
type Result<'a, 'b> =  
  | Ok of 'a      (* result *)  
  | Error of 'b   (* error *)
```

The result type here is
`Result<'a list, string>`

The result type

`Option.get : 'a option -> 'a`

`Option.map : ('a -> 'b) -> 'a option -> 'b option`

`Option.defaultValue : 'a -> 'a option -> 'a`

`Option.bind : ('a -> 'b option) -> 'a option ->
 'b option`

We have seen these before

The result type

`Option.get : 'a option -> 'a`

`Option.map : ('a -> 'b) -> 'a option -> 'b option`

`Option.defaultValue : 'a -> 'a option -> 'a`

`Option.bind : ('a -> 'b option) -> 'a option ->
 'b option`

`Result.map :`

`('a -> 'b) -> Result<'a, 'c> -> Result<'b, 'c>`

`Result.defaultValue :`

`'a -> Result<'a, 'b> -> 'a`

`Result.bind :`

`('a -> Result<'b, 'c>) -> Result<'a, 'c> ->`

`Result<'b, 'c>`

Result.map

```
Option.map :  
  ('a -> 'b) ->  
  'a option ->  
  'b option
```

```
let map f =  
  function  
  | None      -> None  
  | Some x    -> Some (f x)
```

```
Result.map :  
  ('a -> 'b) ->  
  Result<'a, 'c> ->  
  Result<'b, 'c>
```

```
let map f =  
  function  
  | Error err -> Error err  
  | Ok x      -> Ok (f x)
```


Result.defaultValue

Option.defaultValue :
'a -> 'a option -> 'a

```
let defaultValue v =  
  function  
  | None    -> v  
  | Some x  -> x
```

Result.defaultValue :
'a -> Result<'a, 'b> -> 'a

```
let defaultvalue v =  
  function  
  | Error err -> v  
  | Ok x      -> x
```

Result.map

```
Option.bind :  
  ('a -> 'b option) ->  
  'a option ->  
  'b option
```

```
let bind binder =  
  function  
  | None    -> None  
  | Some x  -> binder x
```

```
Result.bind :  
  ('a ->  
   Result<'b, 'c>) ->  
  Result<'a, 'c> ->  
  Result<'b, 'c>
```

```
let bind binder =  
  function  
  | Error err -> Error err  
  | Ok x      -> binder x
```


The result type

```
let safeHead =  
  function  
  | [] -> Error "Cannot take the head of an empty list"  
  | x :: _ -> Ok x  
  
let safeTail =  
  function  
  | [] -> Error "Cannot take the tail of an empty list"  
  | _ :: xs -> Ok xs  
  
[1;2;3;4;5] |> safeTail |> Result.bind safeTail |>  
Result.bind safeTail |> Result.bind safeHead =  
Ok 4
```

The result type

```
let safeHead =  
  function  
  | [] -> Error "Cannot take the head of an empty list"  
  | x :: _ -> Ok x
```

```
let safeTail =  
  function  
  | [] -> Error "Cannot take the tail of an empty list"  
  | _ :: xs -> Ok xs
```

```
[1;2;3] |> safeTail |> Result.bind safeTail |>  
Result.bind safeTail |> Result.bind safeHead =  
Error "Cannot take the head of an empty list"
```

The result type

```
let safeHead =  
  function  
  | [] -> Error "Cannot take the head of an empty list"  
  | x :: _ -> Ok x
```

```
let safeTail =  
  function  
  | [] -> Error "Cannot take the tail of an empty list"  
  | _ :: xs -> Ok xs
```

```
[1;2] |> safeTail |> Result.bind safeTail |>  
Result.bind safeTail |> Result.bind safeHead =  
Error "Cannot take the tail of an empty list"
```


The result type

```
let safeHead =  
  function  
  | [] -> Error "Cannot take the head of an empty list"  
  | x :: _ -> Ok x
```

Note that the first error is propagated

```
| _ :: xs -> Ok xs
```

```
[1;2] |> safeTail |> Result.bind safeTail |>  
Result.bind safeTail |> Result.bind safeHead =  
Error "Cannot take the tail of an empty list"
```

Questions?

Recursive

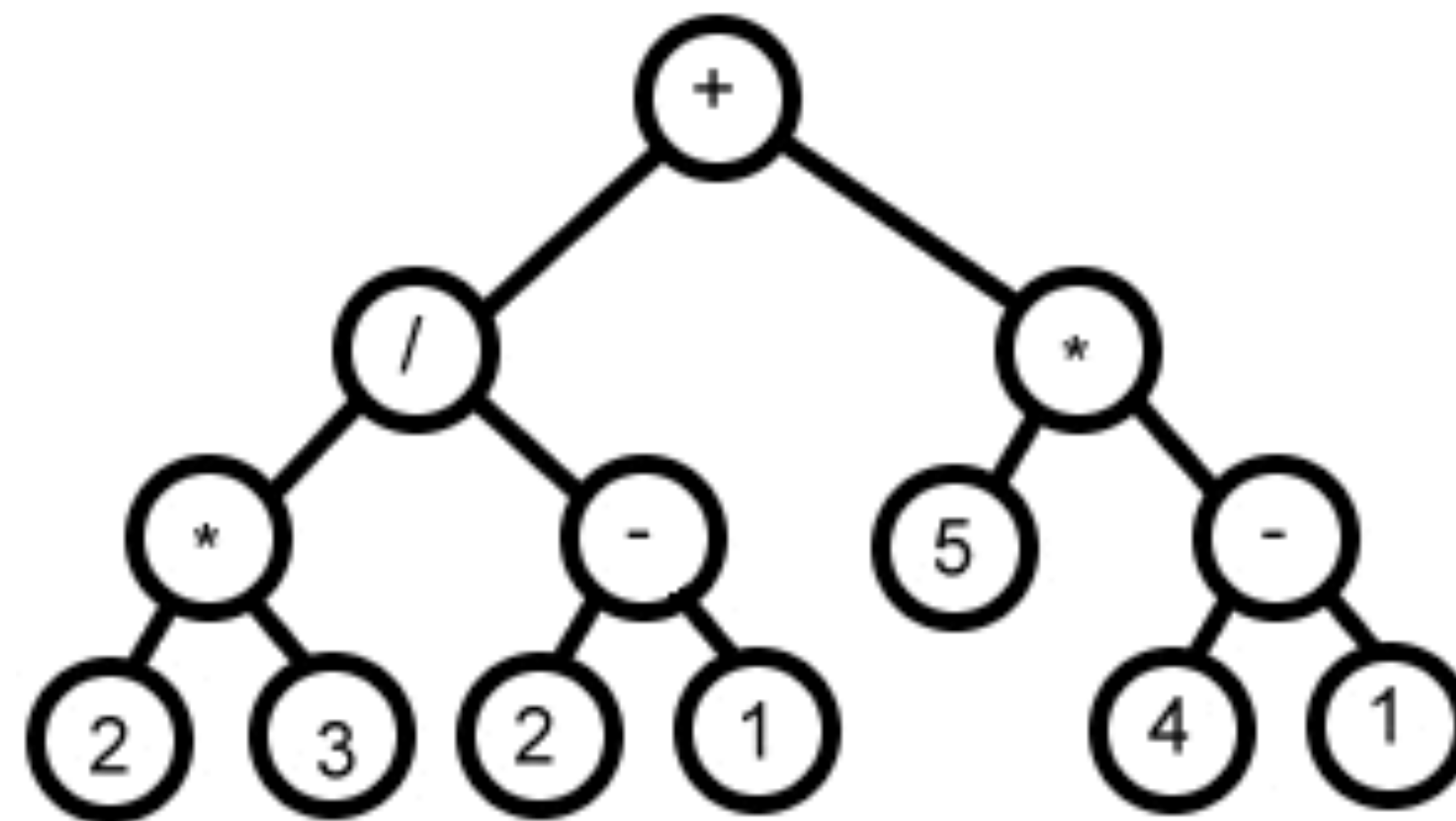
Algebraic types can be recursive and polymorphic

```
type 'a myList =  
  | Nil (* empty list *)  
  | App of ('a * 'a myList) (* cons *)
```

```
let rec length =  
  function  
  | Nil -> 0  
  | App (_, lst) -> 1 + length lst
```

Expression trees

- Expression trees are heavily used by compilers and interpreters
- Nodes contain operators
- Leaves contain values



Expression tree for $2*3/(2-1)+5*(4-1)$

Expression trees

Expression trees are quite easy to code

```
type term =  
  | Const of int  
  | Add of term * term  
  | Sub of term * term  
  | Mul of term * term  
  | Div of term * term
```


Expression trees

Expression trees are quite easy to code

```
type term =  
  | Const of int  
  | Add of term * term  
  | Sub of term * term  
  | Mul of term * term  
  | Div of term * term
```

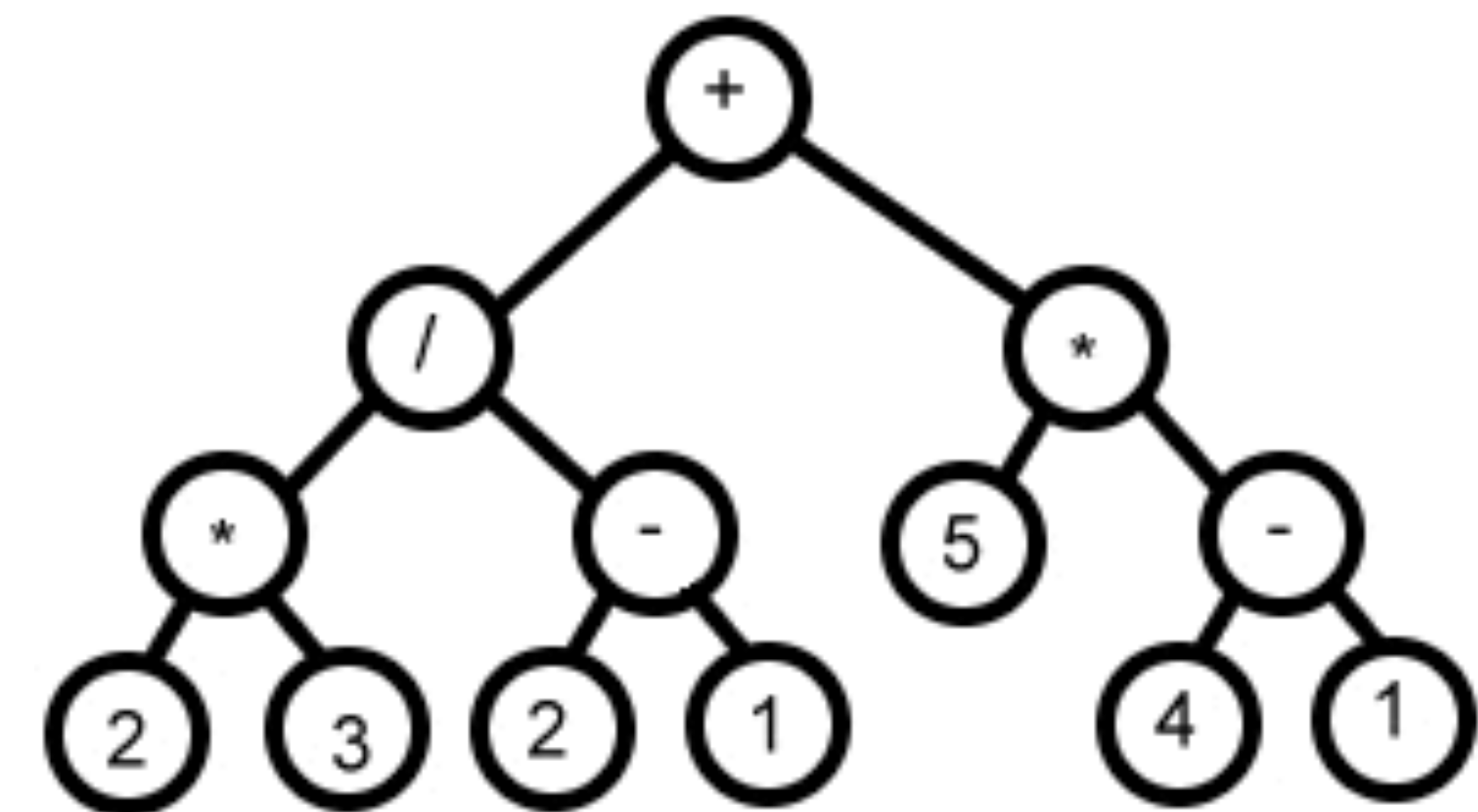
Note that there is no need for parentheses

Expression trees

```

type term =
| Const of int
| Add of term * term
| Sub of term * term
| Mul of term * term
| Div of term * term

```



Expression tree for $2*3/(2-1)+5*(4-1)$

Add

```

      (Div (Mul (Const 2, Const 3),
                Sub (Const 2, Const 1)),
       Mul (Const 5,
            Sub (Const 4, Const 1)))

```


Expression trees

... and also nice to recurse over

```
let rec show : term -> string =  
  function  
  | Const f -> string f  
  | Add (t1, t2) ->  
    "(" + show t1 + " + " + show t2 + ")"  
  | Sub (t1, t2) ->  
    "(" + show t1 + " - " + show t2 + ")"  
  | Mul (t1, t2) ->  
    "(" + show t1 + " * " + show t2 + ")"  
  | Div (t1, t2) ->  
    "(" + show t1 + " / " + show t2 + ")"
```

Expression trees

... and also nice to recurse over

```
let rec show : term -> string =  
  function  
  | Const f -> string f
```

This is one of the green assignments for this week ;)

```
    "(" + show t1 + " - " + show t2 + ")"  
  | Mul (t1, t2) ->  
    "(" + show t1 + " * " + show t2 + ")"  
  | Div (t1, t2) ->  
    "(" + show t1 + " / " + show t2 + ")"
```

Expression Trees

Recall the expression tree from before:

```
let t : term =
```

```
  Add
```

```
    (Div (Mul (Const 2, Const 3),  
            Sub (Const 2, Const 1)),  
      Mul (Const 5,  
            Sub (Const 4, Const 1)))
```

```
> show t
```

```
val it : string =
```

```
"(((2 * 3) / (2 - 1)) + (5 * (4 - 1)))"
```


Evaluating trees

It is also relatively straightforward to compute the value of a tree

```
let rec teval =  
  function  
  | Const x -> x  
  | Add (e1, e2) -> teval e1 + teval e2  
  | Sub (e1, e2) -> teval e1 - teval e2  
  | Mul (e1, e2) -> teval e1 * teval e2  
  | Div (e1, e2) -> teval e1 / teval e2
```

Evaluating trees

```
let t : term =  
  Add  
    (Div (Mul (Const 2, Const 3),  
             Sub (Const 2, Const 1)),  
      Mul (Const 5,  
            Sub (Const 4, Const 1)))
```

```
> teval t  
- val it : int = 21
```


Evaluating trees

```
let t : term =  
  Add  
    (Div (Mul (Const 2, Const 3),  
             Sub (Const 2, Const 2)),  
      Mul (Const 5,  
           Sub (Const 4, Const 1)))
```

> teval t

- **!!! Program crash - division by zero**

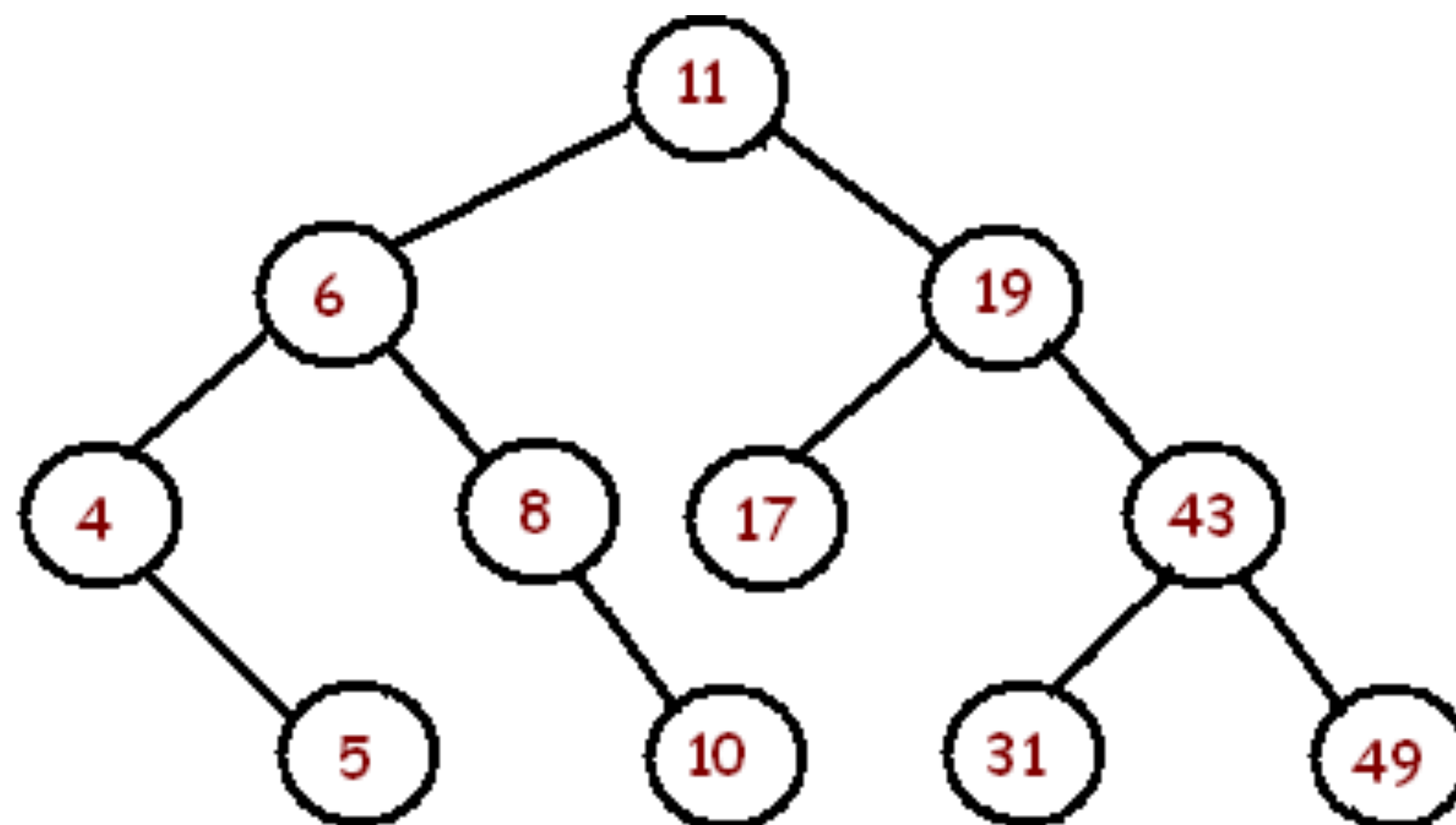
Evaluating trees

Higher-order functions can simplify

```
let rec teval2 =  
  let binop f e1 e2 = f (teval2 e1) (teval2 e2)  
  function  
  | Const x -> x  
  | Add (e1, e2) -> binop (+) e1 e2  
  | Sub (e1, e2) -> binop (-) e1 e2  
  | Mul (e1, e2) -> binop (*) e1 e2  
  | Div (e1, e2) -> binop (/) e1 e2
```

Binary search trees

- All elements in the left subtree are smaller than or equal to the root
- All elements in the right subtree are greater than the root



Let's code
(if there is time)

Questions?